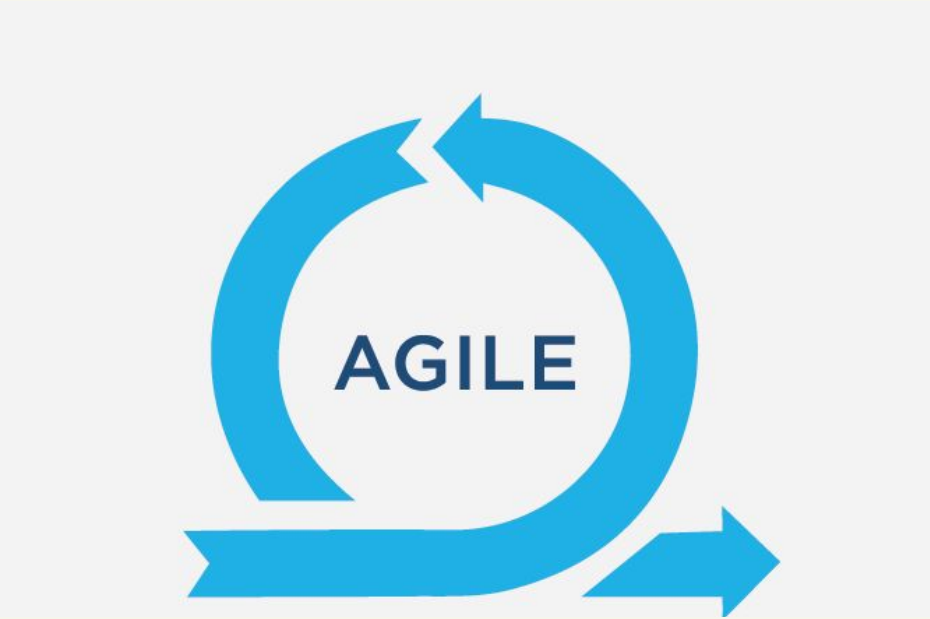
Scrum- Agile vs Waterfall Development

Emily Deleon
CS 250

What is Agile?

- Agile is a flexible approach to software development
- Focuses on teamwork, feedback, and continuous improvement
- Work is done in small cycles called sprints
- Allows changes to be made throughout the project



Agile is basically a flexible way to develop software where the team doesn’t try to plan everything all at once. Instead, the work is broken down into smaller pieces and completed in short cycles called sprints. After each sprint, the team reviews what was built, gets feedback, and makes improvements before moving forward.

Roles on a Scrum-Agile Team

Product Owner

responsible for defining the product vision, creating user stories, and prioritizing work

Scrum Master

supports the team by guiding the Agile process, organizing Scrum events, and removing blockers

Developers

build the product features based on requirements and user storie

Testers/QA

check the product for errors, make sure features work correctly, and help improve quality

Why these roles matter:

Each role has a distinct responsibility, and collaboration between them helps keep the project organized, efficient, and focused on delivering value to the user.

Agile Phases of the SDLC

1 .Requirements/Planning

user stories are created and prioritized based on what the customer needs

2. Design

the team decides how the feature should work and what the interface or process should look like

3. Development

developers build the feature in small parts during a sprint

4. Testing

the feature is checked to make sure it works correctly and meets expectations

5. Deployment/Review

the completed work is reviewed, feedback is gathered, and improvements are planned for the next sprint

6. Maintenance/Iteration

updates and improvements continue as the product evolves

Why these phases matter:

These phases are important because, in Agile, they are not completed just once like in a traditional model but are repeated in smaller cycles throughout the project. This allowed me to continuously improve features in the SNHU Travel project instead of waiting until the end to make changes. For example, when working on features like destination filters and the slide show, I was able to plan, build, test, and then improve those features across multiple sprints. This made it easier to catch issues early and adjust based on feedback without restarting the entire project. Overall, this iterative process made development more manageable, reduced the risk of major errors, and helped ensure that the final product better matched user needs.

Example from the SNHU Travel Project

1 .Requirements/Planning

worked with user stories like adding destination filters, pricing options, and search features so I knew exactly what needed to be built

2. Design

planned how the features would function, like how users would interact with filters and how the slide show would display destinations

3. Development

built the features step by step during each sprint instead of trying to complete everything at once

4. Testing

tested features like search results and filters to make sure they worked correctly and handled different inputs

5. Deployment/Review

reviewed my work after completing features and used feedback to see what needed improvement

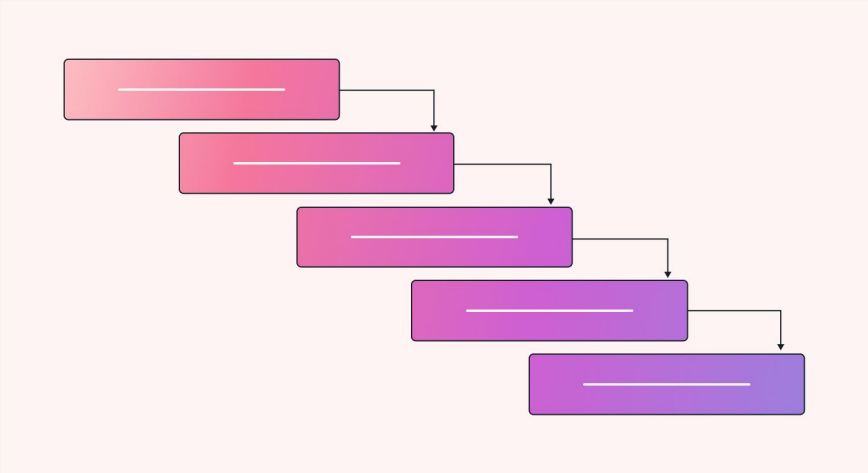
6. Maintenance/Iteration

made updates and improvements in later sprints, like adjusting features after seeing what worked and what didn't

Why this helped:
This process made the project easier to manage because work was broken into smaller parts and could be improved over time.

What is Waterfall?

- Waterfall is a traditional, step-by-step development approach
- Each phase must be completed before moving to the next
- Requirements are set at the beginning and rarely change
- Testing happens near the end of the project



Waterfall is a more traditional approach to software development where everything follows a strict, step-by-step process. Each phase, like planning, design, development, testing, and deployment has to be completed before moving on to the next one.

Waterfall Model

How Waterfall would have been different

- Waterfall follows a strict, step-by-step process:
  requirements → design → development → testing → deployment
- Each phase must be fully completed before moving to the next, with little to no overlap
- Requirements are defined at the beginning and are expected to stay the same
- Changes during development are difficult and often require going back and redoing earlier phases
- Testing happens near the end, so issues may not be discovered until later

Example from the SNHU Travel Project

- If we used Waterfall, all features (like destination filters, pricing options, and slide show design) would have been finalized during the planning phase
- If changes were needed later (like improving how filters worked or updating the slide show layout), the team would have had to go back to the design phase and redo work
- This would have slowed down development and increased the risk of delays
- Also, testing would have happened after everything was built, meaning issues with search or filters might not have been caught early

Agile vs. Waterfall in This Project

Project Comparison

- **Agile allowed continuous updates:**
  Features like destination filters and the slide show could be improved during each sprint instead of being locked in early
- **Waterfall would require rework:**
  If a feature didn't work as expected, the team would have to go back to earlier phases (like design), causing delays
- **Agile supported continuous feedback:**
  Feedback from testing and reviews helped improve features before moving forward
- **Waterfall delays feedback:**
  Issues would not be discovered until the testing phase at the end, making fixes more difficult
- **Agile improved efficiency:**
  Breaking work into smaller tasks made it easier to stay organized and track progress
- **Waterfall increases risk:**
  If requirements were misunderstood early, it could affect the entire project with little flexibility to adjust

Factors to consider when choosing an approach

- Agile is better when:
  - requirements may change
  - feedback is needed often
  - the project benefits from flexibility
- Waterfall is better when:
  - requirements are clear from the start
  - the project is more predictable
  - changes are expected to be minimal

Conclusion

- Scrum-Agile uses clearly defined roles and repeated SDLC phases to support teamwork and continuous progress
- It allows flexibility by breaking work into smaller tasks that can be improved over time
- Waterfall provides a structured and predictable process but limits the ability to make changes once development begins
- In projects where requirements evolve, Waterfall can lead to delays and rework
- For the SNHU Travel project, Agile was more effective because it supported ongoing updates, feedback, and improvements throughout development
- Agile made it easier to adjust features like filters and the slide show without restarting the project

Overall, the Scrum-Agile approach provided more flexibility and efficiency compared to the Waterfall model during the SNHU Travel project. Because the project required updates and adjustments as development progressed, Agile allowed me to make improvements without restarting or delaying the entire process. In contrast, Waterfall would have made these changes more difficult and time-consuming. Based on this experience, Agile is better suited for projects where requirements may change, while Waterfall is more effective for projects with clearly defined and stable requirements.

Reference

Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide*.

Project Management Institute. (2021). *Agile practice guide*.

Rigby, D. K., Sutherland, J., & Takeuchi, H. (2020). *Embracing Agile*. *Harvard Business Review*.

West, D., Gilpin, M., Grant, T., & Anderson, A. (2021). *Waterfall vs. Agile*.